Worker Satisfaction Survey — Data De-identification Workbook

Data De-identification Advisor

July 19, 2026

Contents

Introduction

This workbook guides you through de-identifying the **Apple Grower Satisfaction Survey** (`WorkerSatisfaction_300rows.` 300 rows, 16 columns).

Learning objectives

- Identify privacy risks in survey data
- Apply de-identification steps in R using **dplyr**
- Export a shareable dataset and a separate data key file

Workflow

1. **Setup** — load packages and set file paths
2. **Load and assess** — inspect raw data and review privacy risks
3. **Transform** — apply four de-identification steps (pseudonymization, aggregation, anonymization)
4. **Verify and deliver** — run QA checks and export output files

De-identification steps (Part 3)

Step	Technique	Variables
1	Pseudonymization	`worker_id`, `email_id`, `owner_id`
2	Aggregation	`age` (date of birth → age bands)
3	Pseudonymization	`city`, `orchard_id`
4	Anonymization	`immigration_stat`, `username_id`, `comments`

Output files

- **`WorkerSatisfaction_300rows_deidentified.xlsx`** — for research (e.g. public access. . .)
- **`data_key_file/WorkerSatisfaction_data_key_DUMMY.xlsx`** — for authorized personnel only; store separately from the de-identified data (dummy example)

Do not edit the raw source file (`WorkerSatisfaction_300rows.xlsx`).

Part 1 — Setup

> **R and RStudio (workshop prerequisite)**
> This workbook assumes you have **R** and **RStudio** installed and can open and run `.Rmd` files. If you have not set these up yet, follow the UBC Library Research Commons guide:
> Installing R and RStudio

Required packages

Package	Purpose
`readxl`	Read the original Excel survey file
`dplyr`	Transform data (native pipe `|>`)
`writexl`	Export Excel outputs

Run **Install packages** once if a package is missing.

```
# install.packages("readxl")
# install.packages("dplyr")
# install.packages("writexl")
# install.packages(c("readxl", "dplyr", "writexl"))
```

```
library(readxl)
library(dplyr)
library(writexl)
```

```
DATA_FILE   <- "WorkerSatisfaction_300rows.xlsx"
OUTPUT_FILE <- "WorkerSatisfaction_300rows_deidentified.xlsx"
KEY_DIR     <- "data_key_file"
KEY_FILE    <- file.path(KEY_DIR, "WorkerSatisfaction_data_key_DUMMY.xlsx")
K_THRESHOLD <- 5
```

k-anonymity threshold: categories with fewer than 5 records will be pooled before anonymization.

```
cat("K_THRESHOLD =", K_THRESHOLD, "\n")
```

```
## K_THRESHOLD = 5
```

```
cat("Data key file:", KEY_FILE, "\n")
```

```
## Data key file: data_key_file/WorkerSatisfaction_data_key_DUMMY.xlsx
```

Part 2 — Load and Assess

Before changing anything, we look at the raw survey and ask a simple question of each column: **could this help someone figure out who a respondent is?** This part only *inspects* the data — the actual changes happen in Part 3. For each step below we note what the risk is and how we plan to handle it.

2.1 Load raw data

```
raw <- read_excel(DATA_FILE, sheet = "Unmodified data")

cat("Raw data:", nrow(raw), "rows ×", ncol(raw), "columns\n")
```

```
## Raw data: 300 rows × 16 columns
```

```
head(raw, 10)
```

```
## # A tibble: 10 x 16
##    worker_id        email_id age                immigration_stat city  province
##    <chr>            <chr>    <dttm>             <chr>            <chr> <chr>
##  1 James Strange    james.s~ 1998-09-10 00:00:00 Non-immigrant    Kelo~ B.C.
##  2 Maya Liya        maya.li~ 1994-08-04 00:00:00 Immigrant        Kelo~ B.C.
##  3 Amelio Beal      amelio.~ 1995-01-02 00:00:00 Non-permanent r~ West~ B.C.
##  4 Cara Sahara      cara.sa~ 1991-03-22 00:00:00 Non-permanent r~ West~ B.C.
##  5 Neiv Rieg        neivr@g~ 1990-07-15 00:00:00 Non-permanent r~ Vict~ B.C.
##  6 Troy Ahoy        tahoy@y~ 1988-06-06 00:00:00 Immigrant        Vict~ B.C.
##  7 Dave Mahew       dmahew@~ 2000-11-11 00:00:00 Non-immigrant    Vern~ B.C.
##  8 Jamie Thomas     jamiet@~ 1992-12-18 00:00:00 Immigrant        Vern~ B.C.
##  9 Betty Stills     bettys5~ 1997-09-06 00:00:00 Non-permanent r~ Cobb~ B.C.
## 10 Enrique Iglasias ei123@h~ 1985-10-16 00:00:00 Non-permanent r~ Cobb~ B.C.
## # i 10 more variables: orchard_id <chr>, owner_id <chr>, username_id <chr>,
## #   sns_worked <dbl>, sat_hrs <dbl>, trt_workers <dbl>, trt_manager <dbl>,
## #   cmf_manager <dbl>, sat_work_overall <dbl>, comments <chr>
```

2.2 Step 1 — Direct identifiers (pseudonymization)

Some columns name a person outright — a full name, an email address, or an employer's name.

- **Why it's risky:** these values provide direct identification of individuals, leaving no ambiguity.
- **How we addressed it:** The information is retained in pseudonymized form so researchers can differentiate or link records when appropriate.
- **Restoring original values:** The genuine data are securely kept within a protected data key, allowing only authorized staff to re-identify individuals if absolutely necessary.

Variable	Why it is risky	Becomes
`worker_id`	Full name (300 unique values)	`Worker_001`, …
`email_id`	Email address (300 unique values)	`Email_001`, …
`owner_id`	Orchard owner name (12 unique values)	`Owner_01`, …

2.3 Step 2 — Date of birth (aggregation)

Despite its name, the `age` column actually holds each person's **exact date of birth**.

- **Why it's risky:** a date of birth is effectively a fingerprint — rarely shared by more than a handful of people — so even without names, pairing it with location or workplace could reveal someone's identity.
- **How we addressed it:** We convert each date to a broader **5-year age band** (such as `35-44`). This preserves meaningful patterns for analysis while masking specific ages.

```
data.frame(
  earliest     = as.character(min(as.Date(raw$age))),
  latest       = as.character(max(as.Date(raw$age))),
  unique_dates = length(unique(raw$age))
)
```

```
##     earliest     latest unique_dates
## 1 1970-02-20 2000-11-11          295
```

2.4 Step 3 — Geography and orchards (pseudonymization)

Two columns describe *where* someone is: their `city` and their `orchard_id` (workplace).

- **Why it's risky:** geographic details like city or orchard can sharply narrow down who someone is, especially when combined with age or employer. For instance, living in a small community or working at a particular orchard can make an individual much more identifiable.
- **How we addressed it:** We convert both `city` and `orchard_id` into codes (e.g., `City_01`, `Orchard_01`) so the information remains useful for analysis but not directly identifying. The mapping between codes and original names is stored securely in the data key. We leave `province` unchanged, as all entries are in B.C. and it does not add any re-identification risk.
- **Protecting small groups:** Before assigning codes, we check if any city is too small to be safe on its own. If so, we would pool those into an `"Other BC community"` group to avoid singling anyone out. In this dataset, every city is large enough, so no pooling was required.

```
raw |> count(city, sort = TRUE)
```

```
## # A tibble: 12 x 2
##    city             n
##    <chr>        <int>
##  1 Victoria        33
##  2 Summerland      32
##  3 Keremeos        31
##  4 Kelowna         29
##  5 West Kelowna    29
##  6 Peachland       27
##  7 Cobble Hill     26
##  8 Vernon          24
##  9 Naramata        18
## 10 Oliver          18
## 11 Osoyoos         17
## 12 Penticton       16
```

```
raw |> count(orchard_id, sort = TRUE)
```

```
## # A tibble: 12 x 2
##    orchard_id              n
##    <chr>               <int>
##  1 Billie's Apples        33
##  2 Okanagan Fresh         32
```

```
##  3 Valley View Apples     31
##  4 Applejacks             29
##  5 West Kelowna Apples    29
##  6 Lakeside Groves        27
##  7 Country Apples         26
##  8 Bon Appletit           24
##  9 Golden Valley Farms    18
## 10 Ridgeline Orchard      18
## 11 Peak Harvest Co        17
## 12 Sunrise Orchards       16
```

2.5 Step 4 — Immigration status, handles, and free text (anonymization)

For the last group of columns we use **anonymization** — the change is permanent and there is **no key to undo it**.

`immigration_stat`

- **Why it's risky:** Even though immigration status is not uniquely identifying by itself, some categories might contain just a few respondents. Tiny groups like that can make their members easy to re-identify, especially alongside other information.
- **How we addressed it:** We retain `immigration_stat`, since it offers valuable context for analysis, but proactively scan for any category with fewer than 5 people. If such small-group risks are found, they're merged into a single `"Other immigration status"` group. In this dataset, every immigration group meets the threshold, so no pooling is needed — but the check remains in place to ensure future datasets are always protected in the same way.

```
raw |> count(immigration_stat, sort = TRUE)
```

```
## # A tibble: 3 x 2
##   immigration_stat           n
##   <chr>                  <int>
## 1 Non-permanent resident   107
## 2 Non-immigrant            103
## 3 Immigrant                 90
```

```
rare_immigration <- raw |>
  count(immigration_stat) |>
  filter(n < K_THRESHOLD) |>
  pull(immigration_stat)

cat("Rare immigration groups (n <", K_THRESHOLD, "):\n")
```

```
## Rare immigration groups (n < 5 ):
```

```
rare_immigration
```

```
## character(0)
```

`username_id` (social-media handles)

- **Why it's risky:** a handle (265 unique values here) can often be searched online to find the exact person.
- **Our choice — remove it:** the handles add nothing to the analysis, so we simply delete the column. There's no benefit to keeping even a coded version.

`comments` (free text)

- **Why it's risky:** free-text fields, like comments, may carry indirect identifiers or sensitive personal information. Even after removing or obscuring direct identifiers elsewhere, open responses can include names, workplace details, or circumstantial clues that could lead to re-identification.
- **How we addressed it:** Since it's not possible to automatically guarantee all sensitive details are removed from open text, we opted for full removal of the `comments` field. This ensures no potentially identifying information from free responses remains in the dataset, aligning with best practices for anonymizing qualitative data.

```
raw |>
  filter(comments != "None") |>
  slice(1:10) |>
  select(comments)
```

```
## # A tibble: 10 x 1
##    comments
##    <chr>
##  1 I get that I'm a farm worker and grinding is part of the job, but it'd be ni~
##  2 I don't mind the actual work, but the people I work with and the owners like~
##  3 I only like this job because I can make good money, everything else sucks. I~
##  4 The hours and work are hard, but it's good money and I love the team we have~
##  5 Hard but great work
##  6 I'm an apple picker, it is what it is
##  7 It's a good summer job to help me pay for school, definitely not something I~
##  8 The owner's wife is such a sweetheart, but I don't like the owner himself.  ~
##  9 I hate this job and everybody who works here.  the guys I work with are awfu~
## 10 I don't love the job but I stay for the money and the routine.
```

Part 3 — Transform

Now we make the changes we planned in Part 2, one step at a time. Run the code chunks in order — each step builds on the one before it and prints **10 rows** so you can see the result.

```
step1_ids <- c("worker_id", "email_id", "owner_id")
```

First, scramble the order of the codes (important)

When we turn names into codes, we must be careful *how* the codes are assigned.

- **The trap:** by default, codes get handed out in alphabetical order — `Aiko` becomes `Worker_001`, and so on. Anyone with the shared file could then sort the real names alphabetically and line them up against the codes to guess who is who. That would defeat the whole point.
- **The fix:** we **shuffle** the values first, so the codes come out in a random order (`Aiko` might become `Worker_095`). The mapping is no longer guessable from the data alone — only the key reveals it.
- **Kept consistent:** we build this random order **once** here and reuse it for both the data (below) and the key (Part 4), so the two always match.

  Note: The order is re-shuffled every time you knit the workbook, unless you set a fixed seed in the chunk below.

```
# Uncomment for a permutation that is reproducible across runs; leave commented
# to draw a fresh random ordering each time the workbook is knit.
# set.seed(2024)
```

```r
shuffle_levels <- function(x) {
  vals <- unique(x)
  sample(x = vals, size = length(vals), replace = FALSE)
}

worker_levels  <- shuffle_levels(raw$worker_id)
email_levels   <- shuffle_levels(raw$email_id)
owner_levels   <- shuffle_levels(raw$owner_id)
city_levels    <- shuffle_levels(raw$city)
orchard_levels <- shuffle_levels(raw$orchard_id)

cat("Random pseudonym order - first 5 workers get these codes:\n")
```

```
## Random pseudonym order - first 5 workers get these codes:
```

```r
data.frame(
  original_value = head(worker_levels, 5),
  pseudonym_code = paste0("Worker_", sprintf("%03d", 1:5))
)
```

```
##   original_value pseudonym_code
## 1      Ivan Berg     Worker_001
## 2      Dave Holt     Worker_002
## 3      Rosa Wade     Worker_003
## 4     Finn Mahew     Worker_004
## 5      Finn Voss     Worker_005
```

Step 1: Pseudonymization for direct identifiers

Replace `worker_id`, `email_id`, and `owner_id` with non-descriptive codes. The real values go into the data key (Part 4) so the change can be reversed by authorized staff.

```r
step1 <- raw |>
  mutate(
    worker_id = paste0("Worker_", sprintf("%03d", as.integer(factor(worker_id, levels = worker_levels)))),
    email_id  = paste0("Email_",  sprintf("%03d", as.integer(factor(email_id,  levels = email_levels)))),
    owner_id  = paste0("Owner_",  sprintf("%02d", as.integer(factor(owner_id,  levels = owner_levels))))
  )

cat("Step 1:", nrow(step1), "rows ×", ncol(step1), "columns\n")
```

```
## Step 1: 300 rows × 16 columns
```

```r
head(step1, 10)
```

```
## # A tibble: 10 x 16
##    worker_id  email_id  age                 immigration_stat      city  province
##    <chr>      <chr>     <dttm>              <chr>                 <chr> <chr>
##  1 Worker_165 Email_145 1998-09-10 00:00:00 Non-immigrant         Kelo~ B.C.
##  2 Worker_033 Email_115 1994-08-04 00:00:00 Immigrant             Kelo~ B.C.
##  3 Worker_123 Email_254 1995-01-02 00:00:00 Non-permanent reside~ West~ B.C.
##  4 Worker_238 Email_052 1991-03-22 00:00:00 Non-permanent reside~ West~ B.C.
##  5 Worker_095 Email_034 1990-07-15 00:00:00 Non-permanent reside~ Vict~ B.C.
##  6 Worker_250 Email_099 1988-06-06 00:00:00 Immigrant             Vict~ B.C.
##  7 Worker_057 Email_269 2000-11-11 00:00:00 Non-immigrant         Vern~ B.C.
##  8 Worker_197 Email_139 1992-12-18 00:00:00 Immigrant             Vern~ B.C.
```

```
##  9 Worker_041 Email_069 1997-09-06 00:00:00 Non-permanent reside~ Cobb~ B.C.
## 10 Worker_145 Email_156 1985-10-16 00:00:00 Non-permanent reside~ Cobb~ B.C.
## # i 10 more variables: orchard_id <chr>, owner_id <chr>, username_id <chr>,
## #   sns_worked <dbl>, sat_hrs <dbl>, trt_workers <dbl>, trt_manager <dbl>,
## #   cmf_manager <dbl>, sat_work_overall <dbl>, comments <chr>
```

```
step1 |>
  select(all_of(step1_ids)) |>
  slice(1:5)
```

```
## # A tibble: 5 x 3
##   worker_id  email_id  owner_id
##   <chr>      <chr>     <chr>
## 1 Worker_165 Email_145 Owner_05
## 2 Worker_033 Email_115 Owner_05
## 3 Worker_123 Email_254 Owner_09
## 4 Worker_238 Email_052 Owner_09
## 5 Worker_095 Email_034 Owner_04
```

Step 2: Aggregation for the age variable

Replace each exact date of birth with a **5-year age band**, then drop the original date.

```
to_age_band <- function(dob) {
  yrs <- as.integer(difftime(Sys.Date(), as.Date(dob), units = "days") / 365.25)
  if (yrs < 25)      "18-24"
  else if (yrs < 35) "25-34"
  else if (yrs < 45) "35-44"
  else if (yrs < 55) "45-54"
  else               "55+"
}

step2 <- step1 |>
  mutate(age_band = sapply(age, to_age_band)) |>
  select(-age) |>
  relocate(age_band, .before = immigration_stat)

cat("Step 2:", nrow(step2), "rows ×", ncol(step2), "columns\n")
```

```
## Step 2: 300 rows × 16 columns
```

```
head(step2, 10)
```

```
## # A tibble: 10 x 16
##    worker_id  email_id  age_band immigration_stat      city  province orchard_id
##    <chr>      <chr>     <chr>    <chr>                 <chr> <chr>    <chr>
##  1 Worker_165 Email_145 25-34    Non-immigrant         Kelo~ B.C.     Applejacks
##  2 Worker_033 Email_115 25-34    Immigrant             Kelo~ B.C.     Applejacks
##  3 Worker_123 Email_254 25-34    Non-permanent reside~ West~ B.C.     West Kelo~
##  4 Worker_238 Email_052 35-44    Non-permanent reside~ West~ B.C.     West Kelo~
##  5 Worker_095 Email_034 35-44    Non-permanent reside~ Vict~ B.C.     Billie's ~
##  6 Worker_250 Email_099 35-44    Immigrant             Vict~ B.C.     Billie's ~
##  7 Worker_057 Email_269 25-34    Non-immigrant         Vern~ B.C.     Bon Apple~
##  8 Worker_197 Email_139 25-34    Immigrant             Vern~ B.C.     Bon Apple~
##  9 Worker_041 Email_069 25-34    Non-permanent reside~ Cobb~ B.C.     Country A~
## 10 Worker_145 Email_156 35-44    Non-permanent reside~ Cobb~ B.C.     Country A~
```

```
## # i 9 more variables: owner_id <chr>, username_id <chr>, sns_worked <dbl>,
## #   sat_hrs <dbl>, trt_workers <dbl>, trt_manager <dbl>, cmf_manager <dbl>,
## #   sat_work_overall <dbl>, comments <chr>
```

Step 3: Pseudonymization for city and orchard_id

Replace `city` and `orchard_id` with non-descriptive codes, the same way we did for the direct identifiers in Step 1. These mappings also go into the data key.

```
step3 <- step2 |>
  mutate(
    city       = paste0("City_",    sprintf("%02d", as.integer(factor(city,       levels = city_levels)
    orchard_id = paste0("Orchard_", sprintf("%02d", as.integer(factor(orchard_id, levels = orchard_leve
  )

cat("Step 3:", nrow(step3), "rows ×", ncol(step3), "columns\n")
```

```
## Step 3: 300 rows × 16 columns
```

```
head(step3, 10)
```

```
## # A tibble: 10 x 16
##    worker_id  email_id  age_band immigration_stat      city  province orchard_id
##    <chr>      <chr>     <chr>    <chr>                 <chr> <chr>    <chr>
##  1 Worker_165 Email_145 25-34    Non-immigrant         City~ B.C.     Orchard_07
##  2 Worker_033 Email_115 25-34    Immigrant             City~ B.C.     Orchard_07
##  3 Worker_123 Email_254 25-34    Non-permanent reside~ City~ B.C.     Orchard_04
##  4 Worker_238 Email_052 35-44    Non-permanent reside~ City~ B.C.     Orchard_04
##  5 Worker_095 Email_034 35-44    Non-permanent reside~ City~ B.C.     Orchard_12
##  6 Worker_250 Email_099 35-44    Immigrant             City~ B.C.     Orchard_12
##  7 Worker_057 Email_269 25-34    Non-immigrant         City~ B.C.     Orchard_02
##  8 Worker_197 Email_139 25-34    Immigrant             City~ B.C.     Orchard_02
##  9 Worker_041 Email_069 25-34    Non-permanent reside~ City~ B.C.     Orchard_11
## 10 Worker_145 Email_156 35-44    Non-permanent reside~ City~ B.C.     Orchard_11
## # i 9 more variables: owner_id <chr>, username_id <chr>, sns_worked <dbl>,
## #   sat_hrs <dbl>, trt_workers <dbl>, trt_manager <dbl>, cmf_manager <dbl>,
## #   sat_work_overall <dbl>, comments <chr>
```

```
step3 |> count(city, sort = TRUE)
```

```
## # A tibble: 12 x 2
##    city        n
##    <chr>   <int>
##  1 City_08    33
##  2 City_02    32
##  3 City_09    31
##  4 City_03    29
##  5 City_07    29
##  6 City_12    27
##  7 City_01    26
##  8 City_05    24
##  9 City_10    18
## 10 City_11    18
## 11 City_04    17
## 12 City_06    16
```

```
step3 |> count(orchard_id, sort = TRUE)
```

```
## # A tibble: 12 x 2
##    orchard_id     n
##    <chr>      <int>
##  1 Orchard_12    33
##  2 Orchard_08    32
##  3 Orchard_01    31
##  4 Orchard_04    29
##  5 Orchard_07    29
##  6 Orchard_06    27
##  7 Orchard_11    26
##  8 Orchard_02    24
##  9 Orchard_05    18
## 10 Orchard_09    18
## 11 Orchard_10    17
## 12 Orchard_03    16
```

Step 4: Anonymization for immigration_stat, username_id, and comments

The final step is permanent: we check immigration groups, then delete the handles and comments outright. Nothing here goes into the data key, so **these changes cannot be undone**.

```
# Recompute rare immigration groups here so this step runs on its own,
# without depending on the chunk in Part 2.
rare_immigration <- step3 |>
  count(immigration_stat) |>
  filter(n < K_THRESHOLD) |>
  pull(immigration_stat)

step4 <- step3 |>
  mutate(
    immigration_stat = if_else(
      immigration_stat %in% rare_immigration,
      "Other immigration status",
      immigration_stat
    )
  ) |>
  select(-username_id, -comments)

deid <- step4

cat("Step 4 - final:", nrow(deid), "rows ×", ncol(deid), "columns\n")
```

```
## Step 4 - final: 300 rows × 14 columns
```

```
head(deid, 10)
```

```
## # A tibble: 10 x 14
##    worker_id  email_id  age_band immigration_stat      city  province orchard_id
##    <chr>      <chr>     <chr>    <chr>                 <chr> <chr>    <chr>
##  1 Worker_165 Email_145 25-34    Non-immigrant         City~ B.C.     Orchard_07
##  2 Worker_033 Email_115 25-34    Immigrant             City~ B.C.     Orchard_07
##  3 Worker_123 Email_254 25-34    Non-permanent reside~ City~ B.C.     Orchard_04
##  4 Worker_238 Email_052 35-44    Non-permanent reside~ City~ B.C.     Orchard_04
```

```
##  5 Worker_095 Email_034 35-44    Non-permanent reside~ City~ B.C.     Orchard_12
##  6 Worker_250 Email_099 35-44    Immigrant             City~ B.C.     Orchard_12
##  7 Worker_057 Email_269 25-34    Non-immigrant         City~ B.C.     Orchard_02
##  8 Worker_197 Email_139 25-34    Immigrant             City~ B.C.     Orchard_02
##  9 Worker_041 Email_069 25-34    Non-permanent reside~ City~ B.C.     Orchard_11
## 10 Worker_145 Email_156 35-44    Non-permanent reside~ City~ B.C.     Orchard_11
## # i 7 more variables: owner_id <chr>, sns_worked <dbl>, sat_hrs <dbl>,
## #   trt_workers <dbl>, trt_manager <dbl>, cmf_manager <dbl>,
## #   sat_work_overall <dbl>
```

```
deid |> count(immigration_stat, sort = TRUE)
```

```
## # A tibble: 3 x 2
##   immigration_stat           n
##   <chr>                  <int>
## 1 Non-permanent resident   107
## 2 Non-immigrant            103
## 3 Immigrant                 90
```

What changed

Step	Technique	Variables
1	Pseudonymization	`worker_id` → `Worker_001` …; `email_id` → `Email_001` …; `owner_id` → `Owner_01` …
2	Aggregation	`age` (DOB) → `age_band` (5-year groups)
3	Pseudonymization	`city` → `City_01` …; `orchard_id` → `Orchard_01` …
4	Anonymization	`immigration_stat` pooled if below threshold; `username_id` and `comments` removed

---

Part 4 — Quality Assurance and Data Export

Before sharing anything, we double-check our work, then save two separate files: the safe dataset for analysis, and the protected key for authorized usage only.

4.1 QA checks

These checks confirm nothing identifying slipped through. Every row should read `TRUE` — think of it as a safety checklist before the data leaves your hands.

```
data.frame(
  check = c(
    "Step 1: direct IDs pseudonymized",
    "Step 2: no exact DOB column",
    "Step 3: cities pseudonymized",
    "Step 3: orchards pseudonymized",
    "Step 4: comments removed",
    "Step 4: username_id removed",
    "Row count unchanged"
```

```
  ),
  passed = c(
    !any(unique(raw$worker_id) %in% deid$worker_id),
    !"age" %in% names(deid),
    !any(unique(raw$city) %in% deid$city),
    !any(unique(raw$orchard_id) %in% deid$orchard_id),
    !"comments" %in% names(deid),
    !"username_id" %in% names(deid),
    nrow(deid) == nrow(raw)
  )
)
```

```
##                               check passed
## 1 Step 1: direct IDs pseudonymized   TRUE
## 2      Step 2: no exact DOB column   TRUE
## 3     Step 3: cities pseudonymized   TRUE
## 4   Step 3: orchards pseudonymized   TRUE
## 5         Step 4: comments removed   TRUE
## 6      Step 4: username_id removed   TRUE
## 7              Row count unchanged   TRUE
```

```
data.frame(
  check = c(
    "Emails pseudonymized",
    "Owners pseudonymized"
  ),
  passed = c(
    !any(unique(raw$email_id) %in% deid$email_id),
    !any(unique(raw$owner_id) %in% deid$owner_id)
  )
)
```

```
##                  check passed
## 1 Emails pseudonymized   TRUE
## 2 Owners pseudonymized   TRUE
```

```
data.frame(
  check = c(
    "No rare immigration groups in output",
    "age_band present"
  ),
  passed = c(
    all(table(deid$immigration_stat) >= K_THRESHOLD),
    "age_band" %in% names(deid)
  )
)
```

```
##                                 check passed
## 1 No rare immigration groups in output   TRUE
## 2                     age_band present   TRUE
```

```
data.frame(
  rows               = nrow(deid),
  columns            = ncol(deid),
  smallest_city      = min(table(deid$city)),
  smallest_age_band  = min(table(deid$age_band)),
```

```
  smallest_orchard   = min(table(deid$orchard_id))
)
```

```
##   rows columns smallest_city smallest_age_band smallest_orchard
## 1  300      14            16                14               16
```

Note: Each group should ideally have at least 5 records, or be pooled into a broader category.

4.2 Export de-identified dataset

This is the **safe-to-share** file, ready for analysis. It contains the non-descriptive codes but none of the real names, and it does **not** include the data key.

```
write_xlsx(deid, OUTPUT_FILE)

cat("Exported:", OUTPUT_FILE, "\n")
```

```
## Exported: WorkerSatisfaction_300rows_deidentified.xlsx
```

```
cat("Rows:", nrow(deid), "| Columns:", ncol(deid), "\n\n")
```

```
## Rows: 300 | Columns: 14
```

```
cat("Preview of exported data:\n")
```

```
## Preview of exported data:
```

```
head(deid, 10)
```

```
## # A tibble: 10 x 14
##    worker_id  email_id  age_band immigration_stat      city  province orchard_id
##    <chr>      <chr>     <chr>    <chr>                 <chr> <chr>    <chr>
##  1 Worker_165 Email_145 25-34    Non-immigrant         City~ B.C.     Orchard_07
##  2 Worker_033 Email_115 25-34    Immigrant             City~ B.C.     Orchard_07
##  3 Worker_123 Email_254 25-34    Non-permanent reside~ City~ B.C.     Orchard_04
##  4 Worker_238 Email_052 35-44    Non-permanent reside~ City~ B.C.     Orchard_04
##  5 Worker_095 Email_034 35-44    Non-permanent reside~ City~ B.C.     Orchard_12
##  6 Worker_250 Email_099 35-44    Immigrant             City~ B.C.     Orchard_12
##  7 Worker_057 Email_269 25-34    Non-immigrant         City~ B.C.     Orchard_02
##  8 Worker_197 Email_139 25-34    Immigrant             City~ B.C.     Orchard_02
##  9 Worker_041 Email_069 25-34    Non-permanent reside~ City~ B.C.     Orchard_11
## 10 Worker_145 Email_156 35-44    Non-permanent reside~ City~ B.C.     Orchard_11
## # i 7 more variables: owner_id <chr>, sns_worked <dbl>, sat_hrs <dbl>,
## #   trt_workers <dbl>, trt_manager <dbl>, cmf_manager <dbl>,
## #   sat_work_overall <dbl>
```

4.3 Data key file (DUMMY)

The data key maps pseudonym codes back to real values for **Steps 1 & 3 only**. Because it enables re-identification, store it separately and securely, and never share a real key.

One table (`data_key`) holds every mapping, with columns `variable`, `original_value`, `pseudonym_code`, and `notes`. To restore a field, join its codes back to the original values by `variable`.

Tip: Codes are randomized on each knit, so always regenerate the de-identified file and its key together and keep them as a pair.

File: data_key_file/WorkerSatisfaction_data_key_DUMMY.xlsx

```r
make_key <- function(levels, prefix, width) {
  data.frame(
    original_value = levels,
    pseudonym_code = paste0(prefix, sprintf(paste0("%0", width, "d"), seq_along(levels))),
    stringsAsFactors = FALSE
  )
}

data_key <- bind_rows(
  make_key(worker_levels,  "Worker_",  3)  |> mutate(variable = "worker_id",  notes = NA_character_),
  make_key(email_levels,   "Email_",   3)  |> mutate(variable = "email_id",   notes = NA_character_),
  make_key(owner_levels,   "Owner_",   2)  |> mutate(variable = "owner_id",   notes = NA_character_),
  make_key(orchard_levels, "Orchard_", 2)  |> mutate(variable = "orchard_id", notes = NA_character_),
  make_key(city_levels,    "City_",    2)  |> mutate(variable = "city",       notes = NA_character_)
) |>
  select(variable, original_value, pseudonym_code, notes) |>
  arrange(variable, original_value)

head(data_key, 10)
```

```
##    variable original_value pseudonym_code notes
## 1      city    Cobble Hill        City_01  <NA>
## 2      city        Kelowna        City_07  <NA>
## 3      city       Keremeos        City_09  <NA>
## 4      city       Naramata        City_11  <NA>
## 5      city         Oliver        City_10  <NA>
## 6      city        Osoyoos        City_04  <NA>
## 7      city      Peachland        City_12  <NA>
## 8      city      Penticton        City_06  <NA>
## 9      city     Summerland        City_02  <NA>
## 10     city         Vernon        City_05  <NA>
```

```r
dir.create(KEY_DIR, showWarnings = FALSE)
write_xlsx(list(data_key = data_key), KEY_FILE)
cat("Data key exported:", KEY_FILE)
```

```
## Data key exported: data_key_file/WorkerSatisfaction_data_key_DUMMY.xlsx
```

Closing Summary

Step	Technique	What we did
1	Pseudonymization	Names, emails, owners → codes (key file)
2	Aggregation	Exact birth date → 5-year age band
3	Pseudonymization	City and orchard → codes (key file)
4	Anonymization	Removed handles, comments; pooled tiny groups

The one thing that can still re-identify people is the **data key**. Keep it separate from the shared data, restrict who can open it, and never post it publicly.